
Tutorial: Geolocalización y Gestión de Permisos de Ubicación en Flutter

Este módulo enseña a los estudiantes a interactuar con los sistemas globales de navegación por satélite (GNSS) de los dispositivos móviles, administrar la máquina de estados de los permisos en sistemas operativos modernos y procesar flujos de coordenadas geográficas en tiempo real mediante programación reactiva.

1. Fundamentos Teóricos y Conceptos del Paquete `geolocator`

Para controlar los servicios de ubicación, el paquete `geolocator` actúa como un puente unificado entre las APIs nativas de Android (`FusedLocationProviderClient`) e iOS (`CLLocationManager`). Deben dominar los siguientes conceptos arquitectónicos:

- **Position:** Es el objeto de datos que encapsula las coordenadas geográficas devueltas por el hardware. Contiene propiedades críticas como `latitude` y `longitude` (en grados decimales), `altitude` (metros sobre el nivel del mar), `speed` (velocidad en m/s) y `accuracy` (el radio de incertidumbre en metros).
- **LocationAccuracy (Precisión de Ubicación):** Un enumerador que define qué hardware del teléfono se encenderá. Esto impacta directamente en el consumo energético:
 - `LocationAccuracy.low / lowest`: Usa triangulación de antenas celulares y Wi-Fi (Precisión de ~1 a 10 km). Consumo de batería mínimo.
 - `LocationAccuracy.medium`: Equilibrio basado principalmente en redes y redes Wi-Fi cercanas.
 - `LocationAccuracy.high / best`: Fuerza el encendido del chip **GPS físico** para buscar satélites (Precisión de hasta ~3 metros). Consumo de batería extremadamente alto.
- **LocationSettings (Configuración de Entrada):** Objeto que parametriza el comportamiento del sensor. Permite definir el `distanceFilter` (el umbral en metros que el usuario debe moverse antes de que el sensor emita un nuevo dato) optimizando drásticamente los ciclos de la CPU.
- **LocationPermission (Máquina de Estados de Permisos):** A diferencia de las plataformas tradicionales, los permisos móviles son dinámicos y se dividen en un enum de 5 estados:
 1. `denied`: Permiso denegado por defecto o rechazado una vez; se puede volver a solicitar.
 2. `deniedForever`: El usuario bloqueó el permiso permanentemente. La única forma de activarlo es enviándolo manualmente a los ajustes del sistema.
 3. `whileInUse`: Autorizado solo mientras la aplicación esté abierta en pantalla.
 4. `always`: Autorizado incluso si la app está cerrada o en segundo plano (requiere configuraciones especiales de privacidad).
 5. `unableToDetermine`: El sistema operativo no puede determinar el estado (poco común).

2. Configuración del Entorno (Plataforma Nativa)

El acceso a la ubicación es un dato de privacidad crítico. Es obligatorio declarar las intenciones de rastreo en los manifiestos de cada sistema operativo.

En Android ([android/app/src/main/AndroidManifest.xml](#))

Deben agregarse estas líneas de permisos de hardware justo antes de la etiqueta de apertura `<application>`:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

En iOS ([ios/Runner/Info.plist](#))

Debe añadirse la llave con el mensaje que el sistema operativo le mostrará al alumno al momento de solicitar el permiso:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>Necesitamos acceso a tu ubicación para mostrar las coordenadas en la
práctica de GPS.</string>
```

Estructura de Carpetas del Módulo

Siguiendo la arquitectura limpia por características definida en el proyecto base, los archivos quedan organizados de la siguiente manera:

```
mi_app_sensores/
├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── core/
│   │   └── theme/
│   │       └── theme.dart
│   └── features/
│       ├── home/
│       │   ├── screens/
│       │   └── home_screen.dart
│       └── gps/                                <-- CARPETA DEL MÓDULO ACTUAL
│           ├── screens/
│           └── gps_screen.dart
```

Asegúrate de tener agregada la dependencia en el [pubspec.yaml](#):

```
dependencies:  
  flutter:  
    sdk: flutter  
  geolocator: ^13.0.0
```

3. Código Fuente Completo y Comentado

Archivo `lib/features/gps/screens/gps_screen.dart`

```
import 'package:flutter/material.dart';  
import 'package:geolocator/geolocator.dart';  
import 'dart:async';  
  
class GpsScreen extends StatefulWidget {  
  const GpsScreen({super.key});  
  
  @override  
  State<GpsScreen> createState() => _GpsScreenState();  
}  
  
class _GpsScreenState extends State<GpsScreen> {  
  String _currentPositionText = "Presiona el botón para obtener ubicación";  
  String _livePositionText = "Escuchando cambios en tiempo real...";  
  
  // Objeto de control para abrir, pausar y cerrar el flujo de datos del GPS  
  StreamSubscription<Position>? _positionStreamSubscription;  
  bool _isListening = false;  
  
  // 1. Algoritmo riguroso de control para la máquina de estados de permisos  
  Future<bool> _handlePermission() async {  
    bool serviceEnabled;  
    LocationPermission permission;  
  
    // A. Verificar si el hardware de localización global está encendido en el  
    // teléfono  
    serviceEnabled = await Geolocator.isLocationServiceEnabled();  
    if (!serviceEnabled) {  
      setState(() {  
        _currentPositionText = 'El servicio de ubicación (GPS) está apagado en el  
dispositivo.';  
      });  
      return false;  
    }  
  
    // B. Comprobar el estado actual del permiso otorgado a la app  
    permission = await Geolocator.checkPermission();  
  
    if (permission == LocationPermission.denied) {
```

```

// Solicitar el permiso de forma asíncrona en tiempo de ejecución
permission = await Geolocator.requestPermission();
if (permission == LocationPermission.denied) {
  setState(() {
    _currentPositionText = 'Permiso de ubicación denegado por el usuario.';
  });
  return false;
}
}

// C. Manejar el peor escenario: Bloqueo permanente de permisos
if (permission == LocationPermission.deniedForever) {
  setState(() {
    _currentPositionText = 'Permisos denegados permanentemente. Por favor,
    actívalos desde los Ajustes del sistema.';
  });
  return false;
}

// Si pasa todos los filtros, el acceso al hardware está completamente libre
return true;
}

// 2. Obtener Ubicación Única (Paradigma Request-Response / Future)
Future<void> _getCurrentLocation() async {
  final hasPermission = await _handlePermission();
  if (!hasPermission) return;

  setState(() {
    _currentPositionText = "Conectando con satélites...";
  });

  try {
    // Invocamos el hardware con alta precisión para pruebas de laboratorio
    Position position = await Geolocator.getCurrentPosition(
      locationSettings: const LocationSettings(accuracy: LocationAccuracy.high)
    );

    setState(() {
      _currentPositionText =
        "Latitud: ${position.latitude}\n"
        "Longitud: ${position.longitude}\n"
        "Altitud: ${position.altitude.toStringAsFixed(1)} metros\n"
        "Precisión: ±${position.accuracy.toStringAsFixed(1)} m";
    });
  } catch (e) {
    setState(() {
      _currentPositionText = "Error de hardware: $e";
    });
  }
}

// 3. Escucha en Tiempo Real (Paradigma de Programación Reactiva / Streams)
void _toggleLiveLocation() async {

```

```
final hasPermission = await _handlePermission();
if (!hasPermission) return;

if (_isListening) {
  // Cancelar la suscripción apaga físicamente el hardware receptor de GPS
  await _positionStreamSubscription?.cancel();
  setState(() {
    _isListening = false;
    _livePositionText = "Escucha en tiempo real detenida.";
  });
} else {
  setState(() {
    _isListening = true;
  });

  // Configuración de optimización: Alta precisión, pero solo emite eventos
  // si el dispositivo se desplaza físicamente más de 2 metros
  const LocationSettings locationSettings = LocationSettings(
    accuracy: LocationAccuracy.high,
    distanceFilter: 2,
  );

  // Suscribirse a la tubería de eventos del GPS nativo
  _positionStreamSubscription = Geolocator.getPositionStream(locationSettings:
locationSettings)
    .listen((Position position) {
      setState(() {
        _liveStatusUpdate(position);
      });
    });
}

void _liveStatusUpdate(Position position) {
  // Calculamos velocidad convirtiendo de metros por segundo (m/s) a kilómetros
  // por hora (km/h)
  double speedKmH = position.speed * 3.6;

  _livePositionText =
    "📶 CAMBIO DE POSICIÓN DETECTADO:\n"
    "Lat: ${position.latitude}\n"
    "Lng: ${position.longitude}\n"
    "Velocidad: ${speedKmH.toStringAsFixed(1)} km/h";
}

@override
void dispose() {
  // CRUCIAL: Liberar el Stream para evitar memory leaks y drenado destructivo
  // de batería
  _positionStreamSubscription?.cancel();
  super.dispose();
}

@override
```

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Sensor: GPS y Geolocalización')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          // Panel de Lectura Única
          Card(
            elevation: 4,
            color: Colors.blue.withOpacity(0.05),
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Column(
                children: [
                  const Text('Lectura Única (Bajo Demanda)', style:
TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
                  const SizedBox(height: 12),
                  Text(_currentPositionText, textAlign: TextAlign.center, style:
const TextStyle(fontFamily: 'monospace')),
                  const SizedBox(height: 12),
                  ElevatedButton.icon(
                    onPressed: _getCurrentLocation,
                    icon: const Icon(Icons.location_searching),
                    label: const Text('Obtener Coordenadas'),
                  )
                ],
              ),
            ),
          ),
          const SizedBox(height: 20),

          // Panel de Flujo Continuo
          Card(
            elevation: 4,
            color: _isListening ? Colors.green.withOpacity(0.05) :
Colors.grey.withOpacity(0.05),
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Column(
                children: [
                  const Text('Flujo Continuo (Streams)', style:
TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
                  const SizedBox(height: 12),
                  Text(_livePositionText, textAlign: TextAlign.center, style:
TextStyle(fontFamily: 'monospace', color: _isListening ? Colors.green[800] :
Colors.black)),
                  const SizedBox(height: 12),
                  ElevatedButton.icon(
                    onPressed: _toggleLiveLocation,
                    style: ElevatedButton.styleFrom(
                      backgroundColor: _isListening ? Colors.redAccent :
Colors.teal,

```

```
        foregroundColor: Colors.white,  
      ),  
      icon: Icon(_isListening ? Icons.stop : Icons.play_arrow),  
      label: Text(_isListening ? 'Detener Rastreo' : 'Iniciar  
Rastreo en Vivo'),  
    ),  
  ],  
),  
),  
),  
],  
),  
),  
);  
}  
}
```
